



CONTAINER LIFECYCLE MANAGEMENT SYSTEM

Word Count: 4257(excluding pages 28-43)

Higher Diploma in Science in Software Development

Fergal Hughes

20054847@mydbs.ie

25/09/2026

Contents

1. Introduction	Error! Bookmark not defined.
2. Background and Literature Review	3
3. Specification and Design	9
4. Implementation	18
5. Testing and Results	24
6. Conclusion and Future Work	26
7. References/bibliography.....	28
8. Appendices.....	30

1.1 Introduction

Reusable production containers play an important role in pharmaceutical manufacturing and must be managed to maintain traceability and comply with Good Manufacturing Practice (GMP). Before entering production, containers must be cleaned and approved by Quality Assurance (QA). They may be used for a maximum of fourteen production runs or thirty calendar days, whichever occurs first, before requiring cleaning and Supervisor approval to begin a new lifecycle.

Many organisations still rely on paper records and manual processes to manage reusable production containers. While these methods remain an important part of quality systems, they increase the risk of human error, incomplete traceability, and delays in retrieving information during audits or investigations.

This project presents a full-stack Container Management System designed to work alongside existing manufacturing procedures by providing an additional digital layer of control. The application manages the complete container lifecycle through secure authentication, role-based access control, workflow validation, approval management, movement tracking, QR code scanning, audit logging, and operational dashboards.

The completed system successfully demonstrates the complete lifecycle of reusable production containers, from registration and cleaning through production, lifecycle expiry, Supervisor approval, and reuse.

1.2 Background and Literature Review

Manufacturers operating in regulated industries such as pharmaceutical manufacturing must maintain accurate records and complete traceability throughout the production process. While paper records and manual approvals remain common, they become increasingly difficult to manage as production volumes grow, increasing the risk of human error and incomplete documentation.

Good Manufacturing Practice (GMP) requires organisations to demonstrate that reusable production containers have been cleaned, approved, and used within their approved lifecycle. To support these requirements, manufacturers are increasingly adopting digital systems that improve traceability, workflow management, and regulatory compliance while complementing existing quality procedures.

The Container Management System developed in this project supports this digital transformation by providing an additional layer of control over the container lifecycle. Developed using an Agile Scrum approach, the application records container movements, manages approvals, enforces lifecycle rules, and maintains a complete electronic audit trail for production operators, Quality Assurance (QA) staff, supervisors etc.

1.3 Project Scope and Objectives

The aim of this project is to design and develop a full-stack Container Management System to manage the complete lifecycle of reusable production containers used in pharmaceutical manufacturing. Rather than replacing existing documentation processes, the system provides an additional digital layer of control that enhances traceability, supports regulatory compliance, reduces the risk of human error, and provides greater visibility of container activity throughout the container lifecycle.

The scope of the project includes secure user authentication, role-based access control, container lifecycle management, workflow validation, movement tracking, QA and Supervisor approval workflows, QR code and barcode functionality, operational dashboards, and audit logging. A simulation engine was also developed to generate realistic operational data for system testing, dashboard visualisation, and application evaluation.

The objectives of the project are to:

- Design and implement a relational MySQL database to store container, user, movement, and approval data.
- Develop a RESTful API using Node.js and Express to support the application.
- Create a responsive user interface using React and Material UI.
- Implement business rules to enforce the container lifecycle and approval workflow.
- Provide complete traceability through movement history and audit logging.
- Develop dashboards to monitor container activity and operational performance.
- Generate simulated operational data to support testing and system evaluation.
- Implement QR code and barcode scanning to support rapid container identification and retrieval.

2. Background and Literature Review

2.1 Pharmaceutical Manufacturing

Pharmaceutical manufacturing is one of the most highly regulated industries in the world, requiring organisations to maintain strict control over production processes, equipment, materials, and documentation. Product quality and patient safety depend upon accurate record keeping, validated procedures, and complete traceability throughout the manufacturing lifecycle. As a result, manufacturers must ensure that all equipment and reusable production containers are managed in accordance with established quality procedures and regulatory requirements.

2.2 Good Manufacturing Practice (GMP)

Quality systems are designed to ensure that pharmaceutical products are consistently manufactured to the required standards and are safe for patient use. Good Manufacturing Practice (GMP) provides the framework for achieving this by requiring organisations to follow documented procedures, maintain accurate records, and demonstrate that manufacturing processes remain under control. Regular internal and external audits verify compliance while identifying opportunities for continuous improvement.

As manufacturing processes become more complex, organisations increasingly use digital technologies to strengthen process control, improve traceability, and reduce the risk of human error. Reusable production containers must be cleaned, approved, tracked, and maintained within their defined lifecycle, making accurate record keeping essential for regulatory compliance.

The Container Management System developed in this project supports existing GMP procedures by providing an additional digital layer of control. The application enforces lifecycle rules, records container movements and approvals, maintains a complete audit trail, and provides real-time visibility of container activity, helping organisations strengthen traceability and support continuous improvement.

2.3 Traceability and Audit Trails

Traceability is a fundamental requirement in pharmaceutical manufacturing, allowing organisations to track the history, location, and status of materials, equipment, and reusable production containers throughout the manufacturing process. Accurate records support regulatory compliance while helping to protect product quality, patient safety, and operational accountability.

When a deviation, quality issue, or customer complaint occurs, manufacturers must be able to reconstruct the sequence of events leading to the incident. Audit trails provide this information by recording who performed an action, when it occurred, and the changes that were made. These records support investigations, root cause analysis, and regulatory inspections.

The Container Management System developed in this project improves traceability by maintaining a complete electronic audit trail throughout the container lifecycle. Every container movement, approval, lifecycle event, user, and timestamp is recorded, providing a comprehensive history that supports both operational decision-making and regulatory compliance.

2.4 Digital Workflow Management

Digital workflow management helps organisations improve efficiency, reduce human error, and strengthen regulatory compliance by automating routine tasks and enforcing business rules. Rather than relying solely on manual processes, digital workflows guide users through predefined activities and ensure that each stage is completed before the next can begin.

Within pharmaceutical manufacturing, workflow validation helps ensure that approvals are completed, documentation is available, and operational procedures are followed consistently. This improves process control while supporting compliance with Good Manufacturing Practice (GMP).

The Container Management System developed in this project applies these principles by validating workflow transitions, managing Quality Assurance (QA) and Supervisor approvals, enforcing lifecycle limits based on production use and calendar duration, and maintaining a complete audit trail. Operational dashboards provide users with real-time visibility of container status, pending approvals, and lifecycle activity, supporting informed operational decision-making.

The application has been designed to work alongside existing manufacturing procedures by providing an additional digital layer of control that strengthens traceability, reduces the risk of human error, and supports continuous improvement.

2.5 QR Codes and Digital Identification

QR codes are widely used in manufacturing and warehousing to provide fast and accurate identification of equipment, materials, and assets. By storing a unique identifier, QR codes reduce manual data entry and allow information to be retrieved quickly using a mobile device or scanner.

Within pharmaceutical manufacturing, QR codes improve the identification of reusable production containers and provide rapid access to lifecycle information. This supports day-to-day operations while making it easier to retrieve container records during audits or investigations.

The Container Management System incorporates QR code technology through an integrated Quick Scan feature. Each container is assigned a unique QR code that allows authorised users to instantly access its lifecycle status, movement history, approval records, and operational information. This improves workflow efficiency while supporting the traceability required within a regulated manufacturing environment.

2.6 Dashboard Reporting and Operational Analytics

Operational dashboards provide users with a visual overview of key manufacturing information, allowing them to monitor performance, identify trends, and make informed decisions. By presenting data through charts, summary cards, and Key Performance Indicators (KPIs), dashboards improve operational awareness and provide quick access to important information.

Within pharmaceutical manufacturing, dashboards can support quality management by displaying information such as production activity, pending approvals, and workflow status. This allows users to identify potential issues and monitor operations more effectively.

The Container Management System includes an operational dashboard that displays real-time container status, approval activity, lifecycle utilisation, and movement trends using summary cards and interactive charts. These visualisations provide users with a clear overview of system activity while supporting day-to-day operational monitoring.

2.7 Chapter Summary

This chapter reviewed the background literature relating to pharmaceutical manufacturing, Good Manufacturing Practice, traceability, digital workflow management, QR code technology, operational dashboards, and simulation. The review demonstrates the importance of accurate lifecycle management and provides the theoretical foundation for the design and implementation of the Container Management System described in the following chapters.

3. Specification and Design

3.1 System Requirements

The system requirements were identified during the requirements gathering phase of the project through discussions with personnel familiar with pharmaceutical manufacturing practices, supported by research into Good Manufacturing Practice (GMP) guidelines and existing container management procedures. These requirements formed the basis for the design and implementation of the Container Management System and ensured that the application addressed both operational needs and regulatory considerations.

The requirements were divided into functional requirements, which describe the services and features the system was expected to provide, and non-functional requirements, which define the quality attributes and operational characteristics of the application.

3.1.1 Functional Requirements

The functional requirements describe the core capabilities of the Container Management System. These requirements were identified to ensure the application could effectively support the complete lifecycle of reusable production containers while still maintaining traceability and enforcing business rules.

The principle functional requirements were:

- Secure user authentication
- Role-based access control
- Container registration and management
- Container lifecycle management
- Workflow validation and enforcement
- QA approval workflow
- Supervisor approval workflow
- Automatic lifecycle expiry based on production use and calendar duration
- QR code and barcode scanning for container identification
- Complete movement history and audit logging
- Operational dashboard and analytics
- Simulation engine for operational data generation

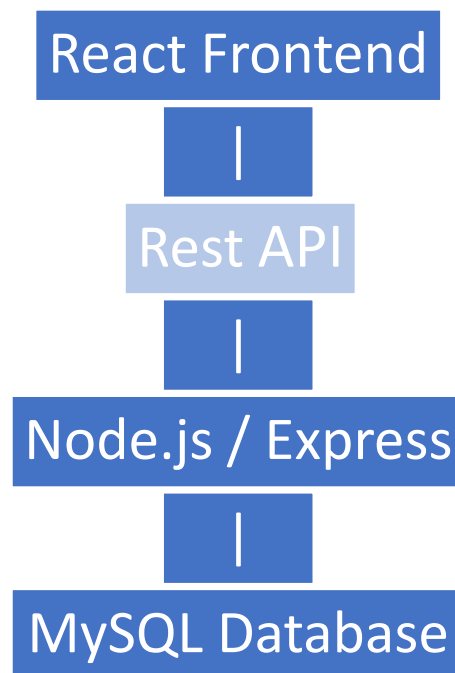
3.1.2 Non- Functional Requirements

In addition to the functional requirements, several non-functional requirements were identified to ensure that the application was reliable, secure, maintainable, and suitable for use within a regulated manufacturing environment.

The key non-functional requirements included:

- **Security** – Authentication and role-based access control to prevent unauthorised access.
- **Reliability** – Accurate enforcement of workflow rules and lifecycle validation.
- **Usability** – A responsive and intuitive user interface suitable for production personnel.
- **Maintainability** – A modular architecture allowing future enhancements and easier maintenance.
- **Performance** – Efficient retrieval of container information and dashboard analytics.
- **Scalability** – A design capable of supporting increased numbers of users, containers, and operational records as organisational requirements grow.

3.2 System Architecture



A three-tier architecture separates the system into presentation, application logic, and data storage layers. This separation makes the system easier to understand, test and evolve over time.

Each tier has a single responsibility:

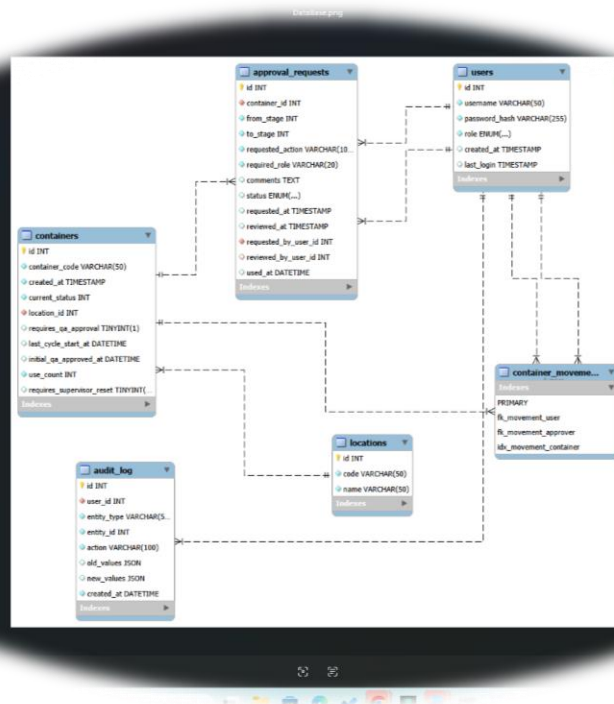
- React Frontend – UI, user interaction, rendering, form handling
- Node.js/Express Api- Business logic, validation, workflow rules, security
- MySQL Database- persistent storage, relational structure, constraints

React and Node.js operate as independent applications communicating through a REST API over HTTP. This three-tier architecture separates the presentation, business logic, and data layers, improving maintainability, scalability, flexibility, and ease of development.

3.3 Database Design

The Container Management System was designed using a relational MySQL database to provide secure storage of operational data while maintaining the relationships required to support container lifecycle management, approval workflows, user authentication, and audit logging. A relational database was selected because it provides data integrity, supports referential constraints, and enables efficient querying of operational information required throughout the application.

Figure 3.3 illustrates the Entity Relationship Diagram (ERD) for the application database. The design centres on the containers table, which stores the current lifecycle information for each reusable production container. Additional tables record users, container movements, approval requests, audit log entries, and location information, allowing complete traceability of every container throughout its operational lifecycle.



Containers

The containers table is the central entity within the database. It stores the current lifecycle status, location, production use count, lifecycle dates, and approval requirements for each reusable production container.

Users

The users table stores authenticated users and their assigned roles, including Operator, Quality Assurance (QA), Supervisor, and Administrator. These roles are used to enforce role-based access control throughout the application.

Approval Requests

The approval_requests table records QA and Supervisor approval activities, including the requested action, approval status, reviewer, timestamps, and comments.

Container Movements

The container_movements table records every movement between workflow stages, providing a complete history of each container's lifecycle and forming the application's electronic audit trail.

Audit Log

The audit_log table records significant user actions and system events, providing additional accountability and supporting operational investigations.

Locations

The locations table stores the physical workflow locations, such as Production, Cleaning, and Clean Storage, improving data consistency throughout the application.

Database Relationships

As shown in Figure 3.3, the containers table forms the centre of the database, with related movement, approval, audit, and user records linked through foreign key relationships. This structure maintains integrity while supporting complete lifecycle traceability.

3.4 Workflow Design

Lifecycle steps	Received
	Cleaning
	Clean Storage
	QA approval
	Production
	Cleaning
	Supervisor approval
	Clean Storage
	Production

The Business rules:

- QA approval before first production run, a new container
- Maximum Fourteen production runs
- Maximum 30 calendar days
- Automatic lifecycle expiry
- Supervisor approval after expiry before reuse
- Complete movement history maintained

3.5 User Interface Design

The frontend was developed using React and Material UI to provide a responsive and easy-to-use interface. Material UI components such as cards, tables, and navigation menus were used to present container information consistently across desktop and mobile devices. Data tables support searching and filtering, while the integrated Quick Scan feature allows users to retrieve container information quickly using QR codes.

3.6 Security and Design

JWT (JSON Web Token) is used to authenticate users after login. When a user logs in successfully, the server generates a token containing the user's identity and role. The frontend sends this token with each request, allowing the server to verify the user's identity and control access to protected resources without storing session data.

User passwords are protected using bcrypt hashing. Instead of storing passwords as plain text, they are converted into a secure hashed value before being saved in the database, helping to protect user accounts if the database is ever compromised.

Role-Based Access Control (RBAC)

RBAC means different users have different permissions. For example:

- Normal user can scan containers and update workflows.
- QA and supervisors can approve containers

The role is stored inside the JWT token. The backend checks the role before allowing access to certain routes. This stops users from doing actions they are not allowed to do.

Protected API Routes

Some API routes require a valid JWT token. If the token is missing or invalid, the server blocks the request. This protects sensitive data like container details, movement history etc. Only logged-in users can access protected routes.

Permission Checking

Permission checking makes sure the user has the right role before performing an action. The backend checks both the token and the role before running the action. This prevents mistakes and keeps the system secure.

3.7 Dashboard and Analytics Design

KPI Cards

KPI cards Examples:

- Total containers
- Containers needing QA
- Containers in production
- Containers that are expired

These cards help users understand the current situation in just a few seconds. They make the dashboard simple and easy to read.

Charts

Charts help users see patterns instead of just numbers.

- How many containers moved each day
- How many QA approvals happened this week
- How many containers reached expiry

Charts make trends easier to understand and help users spot unusual activity.

Analytics

Analytics give deeper insight into how the system is performing. They help answer questions like:

- Are containers moving too slowly?
- Is QA approval taking too long?
- Are certain locations overloaded?

Analytics turn raw data into useful information that helps improve operations.

Movement Trends

Movement trends show how containers move over time. This helps users see:

- Busy periods
- Slow periods
- Repeated patterns
- Possible bottlenecks

Understanding movement trends helps managers plan better and avoid delays.

Lifecycle Monitoring

Lifecycle monitoring shows where each container is in its workflow:

- Cleaning
- Clean storage
- QA approval
- Production
- Expired
- Reset

This helps staff know which containers are ready, which need attention, and which are close to expiry. It reduces mistakes and keeps the process running smoothly.

3.8 Chapter summary

This chapter described the specification and design of the Container Management System, including the functional and non-functional requirements, system architecture, database design, workflow design, user interface, security, and dashboard components. These design decisions provided the foundation for the implementation described in the following chapter.

4. Implementation

4.1 Development Environment

The Container Management System was implemented as a full-stack web application using modern JavaScript technologies. The application was developed using Visual Studio Code as the primary development environment, with Git and GitHub providing version control throughout the project. The frontend was implemented using React and Material UI to provide a responsive user interface, while the backend was developed using Node.js and Express to expose a RESTful API. MySQL was selected as the relational database due to its reliability, support for referential integrity, and compatibility with the chosen technology stack.

Technology	Purpose
React	Frontend user interface
Material UI	User interface components
Node.js	Backend runtime
Express.js	REST API
MySQL	Database
JWT	Authentication
Chart.js	Dashboard visualisation
Jest	Unit testing
GIT/GitHub	Version control
Thunder Client	API testing
MySQL Workbench	Database management

4.2 Backend Implementation

The backend was implemented using Node.js and Express.js following a layered architecture to improve maintainability and separation of responsibilities. The application was organised into controllers, services, models, routes, and middleware, with each layer performing a specific role. Controllers handled HTTP requests, services implemented the business logic, models managed database interactions, routes defined the API endpoints, and middleware provided authentication and request validation. This modular structure simplified development, testing, and future maintenance.

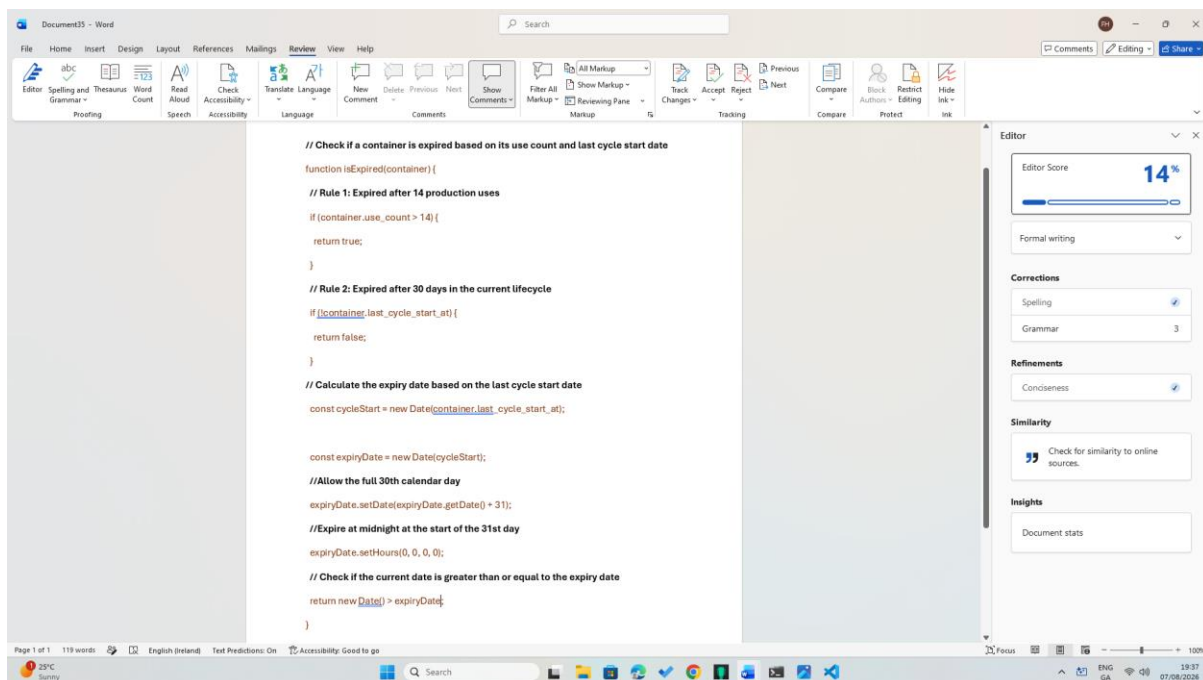
4.3 Authentication and Authorisation

User authentication was implemented using JSON Web Tokens (JWT) to provide secure access to the application. Passwords were securely stored using hashing techniques, while role-based access control restricted access to authorised users. Middleware was used to validate authentication tokens and ensure that only users with the appropriate roles could perform actions such as approving workflows or managing container movements.

4.4 Container Lifecycle Engine

The container lifecycle engine formed the core of the application by enforcing the business rules governing reusable production containers. Workflow validation ensured that containers progressed through the correct lifecycle stages, while lifecycle limits were enforced based on a maximum of fourteen production uses or thirty calendar days, whichever occurred first. Containers reaching either limit were automatically removed from production, transferred to the cleaning stage, and required Supervisor approval before beginning a new lifecycle. Initial entry into production also required Quality Assurance approval. The snippet below illustrates the lifecycle validation logic implemented within the application.

Code snippet: Expiry business rule 14 uses or 30 calendar days

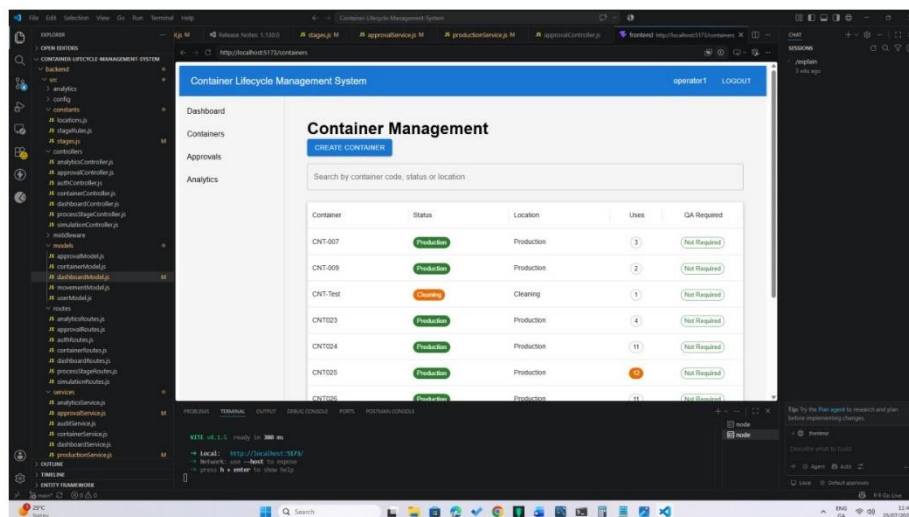
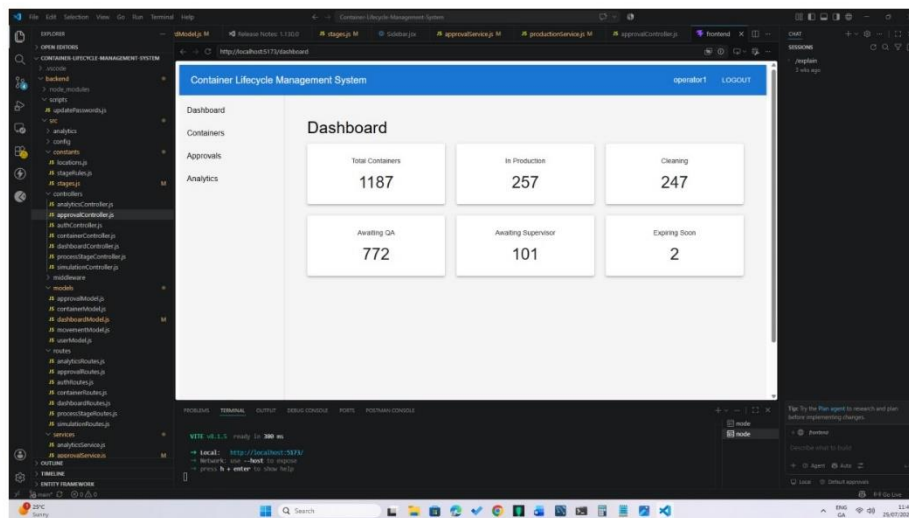


4.5 Approval workflow

The approval workflow was implemented to ensure that critical lifecycle transitions were authorised by the appropriate personnel. Initial production required Quality Assurance approval, while containers completing a lifecycle required Supervisor approval before reuse. Approval requests were generated automatically when required, validated against user roles, and recorded within the database to provide a complete audit trail of all approval activities.

4.6 Dashboard and Analytics

The application includes an operational dashboard that provides users with real-time visibility of container activity through summary cards, charts, and analytical reports. Dashboard information is generated using SQL aggregation queries and presented using Chart.js to display operational metrics, lifecycle usage, approval status, and movement trends. These visualisations assist users in monitoring system activity and identifying containers approaching lifecycle limits.



4.7 QR Code Implementation

Each container is assigned a unique QR code to support rapid identification and retrieval of container information. The integrated Quick Scan feature allows authorised users to scan a container and immediately access its lifecycle status, approval history, and movement records. This functionality reduces manual searching and improves operational efficiency.



Scanning_Container_code.mp4

4.8 Simulation Engine

A simulation engine was developed to generate realistic operational data for testing and evaluation. The simulator automatically processed container movements, approval workflows, and production activities, allowing the complete lifecycle to be exercised repeatedly. This provided sufficient data to populate the dashboard and validate the application's workflow under realistic operating conditions.

4.9 Testing Support Features

Several tools were used throughout development to support testing and validation. Jest was used for unit testing, Thunder Client for API testing, MySQL Workbench for database verification, and the simulation engine for generating operational data and validating lifecycle behaviour under repeated execution.

4.10 Chapter Summary

This chapter described the implementation of the Container Management System, including the backend architecture, authentication, lifecycle engine, approval workflow, dashboard, QR code functionality, and simulation engine. Together, these components formed a complete web application capable of managing the lifecycle of reusable production containers while enforcing business rules and supporting regulatory compliance.

5. Testing and Results

5.1 Testing Strategy

The Container Management System was tested throughout development using a combination of unit testing, API testing, manual functional testing, and end-to-end workflow validation. Testing was carried out incrementally as new functionality was implemented to ensure that individual components and complete business workflows operated as expected.

5.2 Unit and API Testing

Unit tests were developed using Jest to verify the correctness of the application's business logic. REST API endpoints were tested using Thunder Client to validate request handling, authentication, workflow processing, and error handling. Database queries were verified using MySQL Workbench to ensure that lifecycle data, approvals, and movement records were correctly stored and retrieved.

5.3 Functional Testing

Functional testing focused on validating the complete lifecycle of reusable production containers. The following business rules were successfully verified:

- User authentication and role-based access control.
- Container registration and movement between workflow stages.
- Initial Quality Assurance approval before first production use.
- Automatic lifecycle expiry after fourteen production uses or thirty calendar days.
- Supervisor approval before beginning a new lifecycle.
- QR code scanning and container retrieval.
- Dashboard reporting and operational analytics.

5.4 Results

The completed application successfully demonstrated the complete lifecycle of reusable production containers from initial registration through cleaning, approval, production use, lifecycle expiry, Supervisor approval, and reuse. Dashboard analytics correctly reflected operational activity generated by the simulation engine, while unit and API testing confirmed that the workflow engine consistently enforced the defined business rules.

5.5 Critical evaluation

The project successfully achieved its primary objective of developing a digital Container Management System capable of supporting traceability, workflow validation, and regulatory compliance. The layered architecture improved maintainability, while the simulation engine enabled realistic operational testing and dashboard evaluation.

One limitation of the project is that operational analytics were generated using simulated rather than live manufacturing data. Although preliminary regression analysis was explored, the available dataset was not considered sufficiently representative to support meaningful predictive modelling. This has been identified as an opportunity for future development as larger volumes of operational data become available.

5.6 Chapter Summary

This chapter presented the testing strategy and evaluation of the Container Management System. The results demonstrated that the application successfully implemented the required business rules, approval workflows, lifecycle management, and operational reporting, confirming that the system met the objectives defined for the project.

6. Conclusion and Future Work

6.1 Conclusion

The aim of this project was to design and develop a full-stack Container Management System to manage the lifecycle of reusable production containers within a pharmaceutical manufacturing environment. The completed application improves traceability, supports regulatory compliance, and provides an additional digital layer of control alongside existing manufacturing procedures.

The system includes secure authentication, role-based access control, lifecycle management, workflow validation, QA and Supervisor approvals, QR code scanning, audit logging, operational dashboards, and a simulation engine. Business rules governing container usage and lifecycle expiry are automatically enforced while maintaining a complete history of movements and approvals.

Testing confirmed that the application successfully implemented the required workflow and lifecycle management processes. Automated and manual testing verified that business rules, user permissions, and lifecycle transitions operated as expected.

6.2 Project Objectives

The objectives defined at the beginning of the project were successfully achieved. A relational MySQL database was developed to support the storage of container, user, movement, and approval data, while a RESTful API implemented using Node.js and Express provided secure communication between the frontend and backend. A responsive React user interface was developed to provide an intuitive user experience, supported by operational dashboards, QR code scanning, and analytical reporting. The simulation engine further demonstrated the application's ability to generate realistic operational data for testing and evaluation.

6.3 Reflection

The project provided valuable experience in the design and development of a complete full-stack software application. Throughout the development process, knowledge gained during the Higher Diploma programme was applied to database design, backend development, frontend development, authentication, software testing, and Agile project management.

One of the most significant learning experiences was implementing and refining the container lifecycle workflow. Developing business rule validation, approval workflows, and lifecycle enforcement required careful consideration of both software engineering principles and real-world manufacturing practices. The iterative Agile approach allowed features to be developed, tested, and refined throughout successive sprints, resulting in a more robust and maintainable application.

6.4 Future Work

The completed application provides a strong foundation for future development. Potential enhancements include integration with Manufacturing Execution Systems (MES) or Enterprise Resource Planning (ERP) platforms, automated email or in-application notifications for pending approvals and lifecycle expiry and further expand reporting capabilities.

I attempted using regression analysis during the project, but the dataset was too small and not reliable enough to build useful predictions. When more operational data is collected in the future, the system could add regression and machine-learning models to predict container usage, expiry, and trends. This would make the system even better for decision-making.

7. References / Bibliography

- Agency, E. (2026, June 30). *Guidance on good manufacturing practice and good distribution practice: Questions and answers*. Retrieved from European Medicines Agency(EMA): <https://www.ema.europa.eu/>
- Auth0. (2026, June 29). *Introduction to JSON Web Tokens*. Retrieved from [jwt.io](https://jwt.io/introduction): <https://jwt.io/introduction>
- by Alec Sharp, a. P. (2008). *Workflow Modeling: Tools for Process Improvement and Application Development, Second Edition*. Artech House.
- Cairo, A. (2019). How Charts Lie: Getting smarter about Visual Information. In A. Cairo, *Getting Smarter about Visual Information* (pp. 21-36). W. W. Norton & Company, Incorporate.
- Chart.js. (2026, June 30). *Chart.js*. Retrieved from [Charts.js|Chart.js](https://chartjs.org): <https://chartjs.org>
- Devanshi Bhatt, K. P. (2023). ReactJS: A Comprehensive Analysis of its features, Performance, and Suitability for Modern Web Development. *IJSREM Journal*, 1-7.
- Ducellier, N. M. (2016). *Traceability and Capitalization*. London: John Wiley & Sons, Incorporated.
- Express.js. (2026, June 28). <https://expressjs.com/en/>. Retrieved from [Express.js](https://expressjs.com): <https://expressjs.com>
- Ghansham Jadhav1, F. G. (2020, june). Role of Node.js in Modern Web Application Development. *International Research Journal of Engineering and Technology(IRJET)*, 6145-6150. Retrieved from <https://www.irjet.net>
- Hoque, S. (2020). *Learn MERN Stack Development by Building Modern Web Apps Using MongoDB, Express, React, and Node. js*. BIRMINGHAM - MUMBAI: Packt Publishing, Limited.
- Jest. (2026, July 20). *Jest* . Retrieved from [Jest Documentation](https://jestjs.io/docs/getting-started): <https://jestjs.io/docs/getting-started>
- MDN Web Docs. (2026, June 29). *Express web framework (Node.js/JavaScript)*. Retrieved from [MDN Web Docs](https://developer.mozilla.org): <https://developer.mozilla.org>
- MUI. (2026, July 24). *Material UI Documentation*. Retrieved from [MUI](https://mui.com): <https://mui.com>
- MySQL. (2026, June 30). *MySQL Documentation*. Retrieved from [MySQL](https://dev.mysql.com): <https://dev.mysql.com>
- Node.js. (2026, June 28). *Node.js API Documentation*. Retrieved from [Node.js](https://nodejs.org): <https://nodejs.org>
- React. (2026, June 27). *React.dev*. Retrieved from [React](https://react.dev): <https://react.dev>

- Samarasinghe, A. (2026, June 30). *Add Charts to Your Website Using Chart.js*. Retrieved from Better Programming: <https://betterprogramming.pub>
- Siegel, E. (2016). *The power to predict who will CLICK, BUY, LIE, OR DIE*. New Jersey: John Wiley and sons .
- Villanueva, J. A. (2026, april 10). *A Systematic Literature Review of Workflow Efficiency in Software Development Through the Kanban Framework*. Retrieved from ACM Digital Library: <https://doi.org/10.1145/3779657.3779666>
- Williams, S. T. (2007). Learning MySQL. In S. T. Williams, *Learning MySQL* (pp. 109-133). California: O'Reilly Media.

Appendix A-Product Backlog

ID	Product Backlog Item	Priority (MoSCoW)	Story Points	Status
PB01	Project Planning and Documentation	Must Have	3	Completed
PB02	Establish Project Structure	Must Have	2	Completed
PB03	User Authentication	Must Have	5	Completed
PB04	Role-Based Access Control	Must Have	5	Completed
PB05	Register Container	Must Have	5	Completed
PB06	View Containers	Must Have	8	Completed
PB07	Move Containers Through Workflow	Must Have	8	Completed
PB08	QA Approval Workflow	Must Have	8	Completed
PB09	Supervisor Approval Workflow	Must Have	8	Completed
PB10	Production Lifecycle Management	Must Have	8	Completed
PB11	Lifecycle Validation	Must Have	5	Completed
PB12	Production Use Tracking	Must Have	5	Completed
PB13	Lifecycle Reset	Must Have	5	Completed
PB14	30-Day Expiry Rule	Must Have	5	Completed
PB15	Audit Trail	Must Have	3	Completed
PB16	Dashboard Summary	Must Have	8	Completed
PB17	Dashboard Charts	Should Have	5	Completed
PB18	QR and Barcode Scanning	Should Have	5	Completed
PB19	Automated Container Simulation	Should Have	8	Completed
PB20	Integration Testing	Must Have	3	Completed

The Product Backlog was prioritised using the MoSCoW technique. Core functionality was classified as Must Have, while enhancements such as dashboard charts, QR/barcode scanning, and simulation were classified as Should Have. Story points were assigned to estimate development effort and support sprint planning. All planned functionality included in the final project scope was completed successfully.

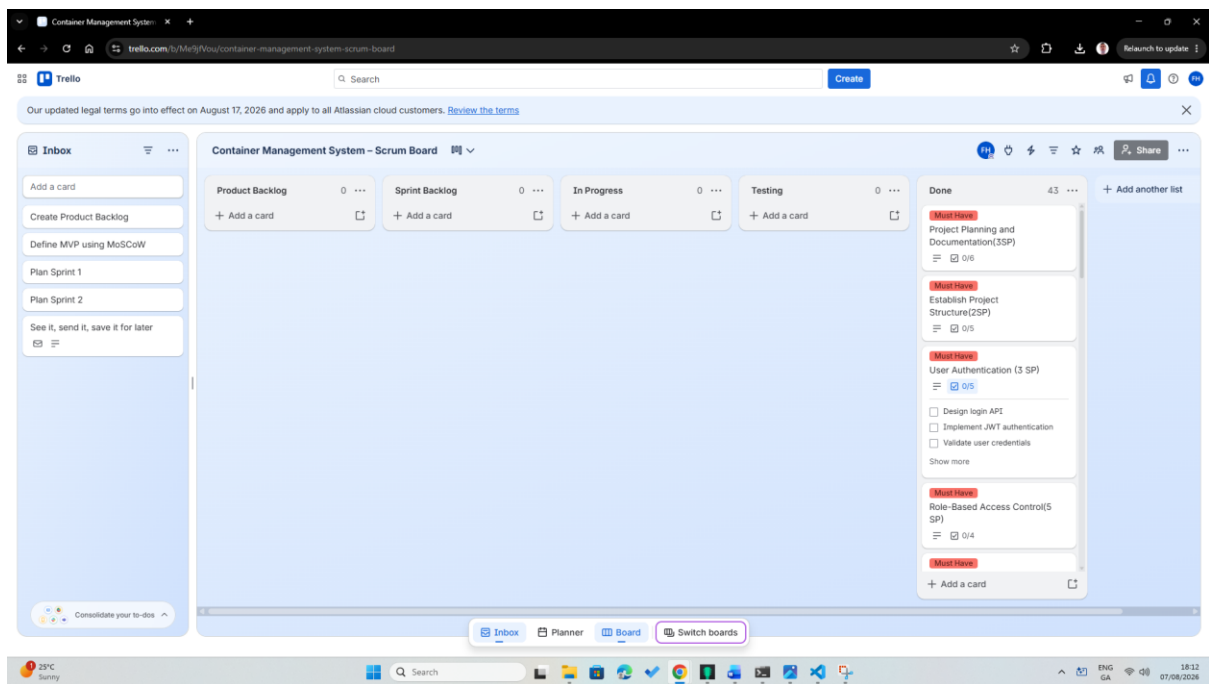
Appendix B – Sprint Planning

Sprint	Duration	Objective	Outcome
Sprint 1	Week 1,2	Project planning, requirements gathering and database design	Completed
Sprint 2	Week 3,4	Backend development, authentication and database implementation	Completed
Sprint 3	Week 5,6	Workflow engine, container lifecycle and approval logic	Completed
Sprint 4	Week 7,8	Testing, simulation engine and backend refinement	Completed
Sprint 5	Week 9,10	React frontend, dashboard and system integration	Completed
Sprint 6	Week 11,12	Analytics, Quick Scan, lifecycle validation and final testing	Completed

Sprint Deliverables

Sprint	Key Deliverables
Sprint 1	Requirements analysis, proposal, ERD, project setup
Sprint 2	JWT authentication, REST API, MySQL database
Sprint 3	QA workflow, Supervisor workflow, lifecycle engine
Sprint 4	Simulation engine, Jest testing, workflow validation
Sprint 5	React interface, dashboards, container management
Sprint 6	Analytics, QR/Barcode scanning, lifecycle completion, final testing

Trello Board showing completed Backlog items

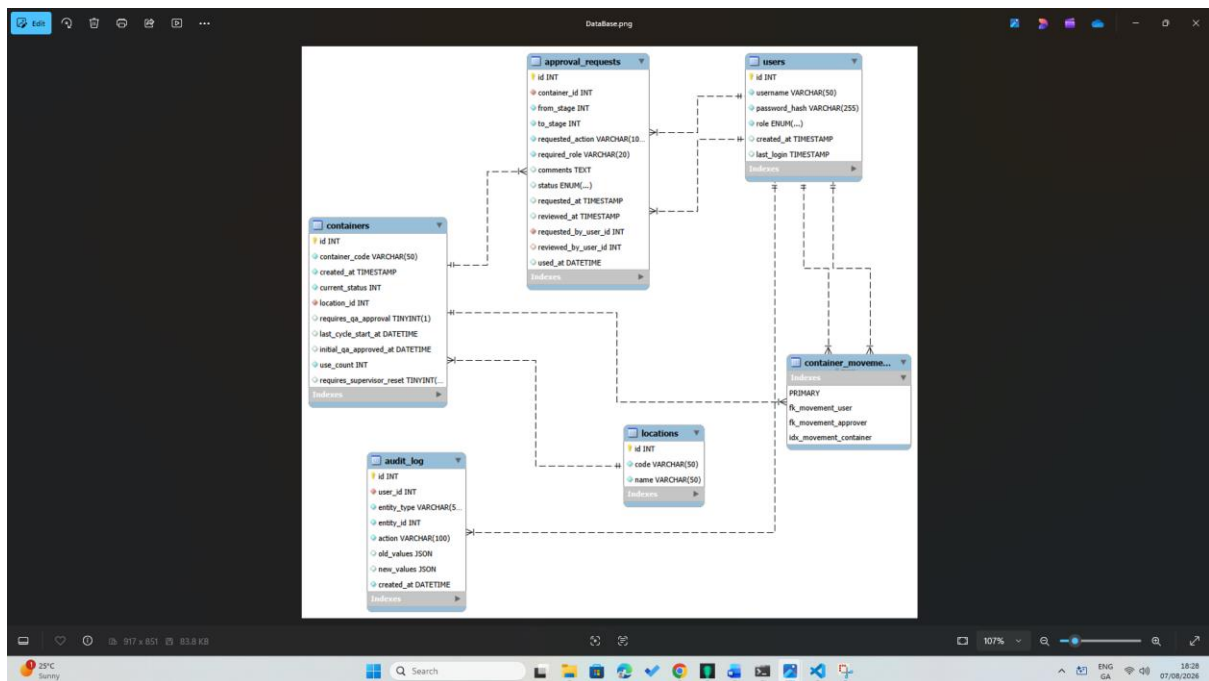


Appendix C – System Design

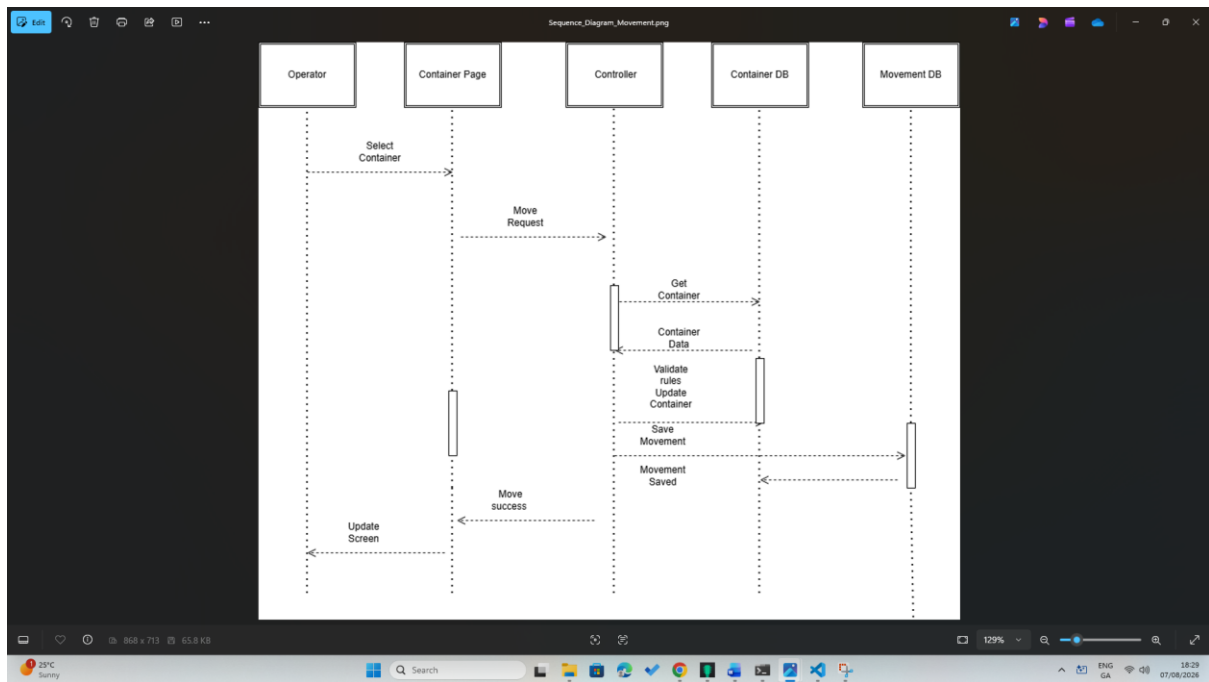
Description

The application follows a three-tier architecture consisting of a React frontend, a Node.js/Express backend, and a MySQL database. This architecture separates presentation, business logic, and data storage, improving maintainability and scalability.

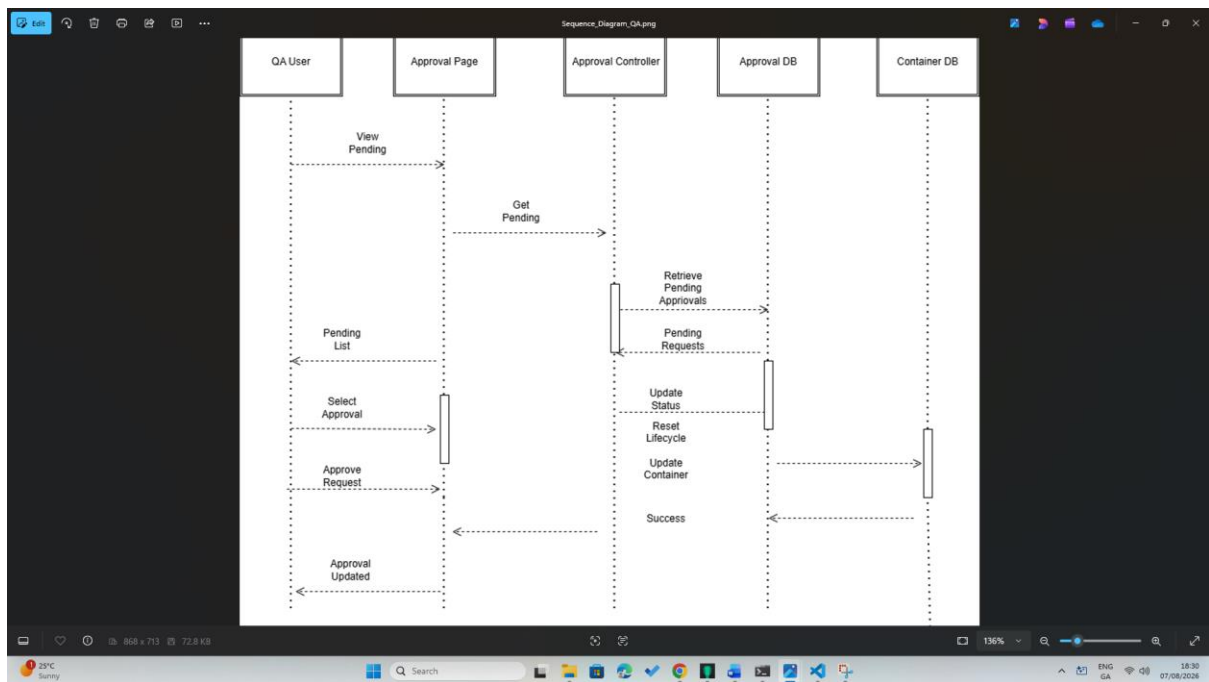
ERD



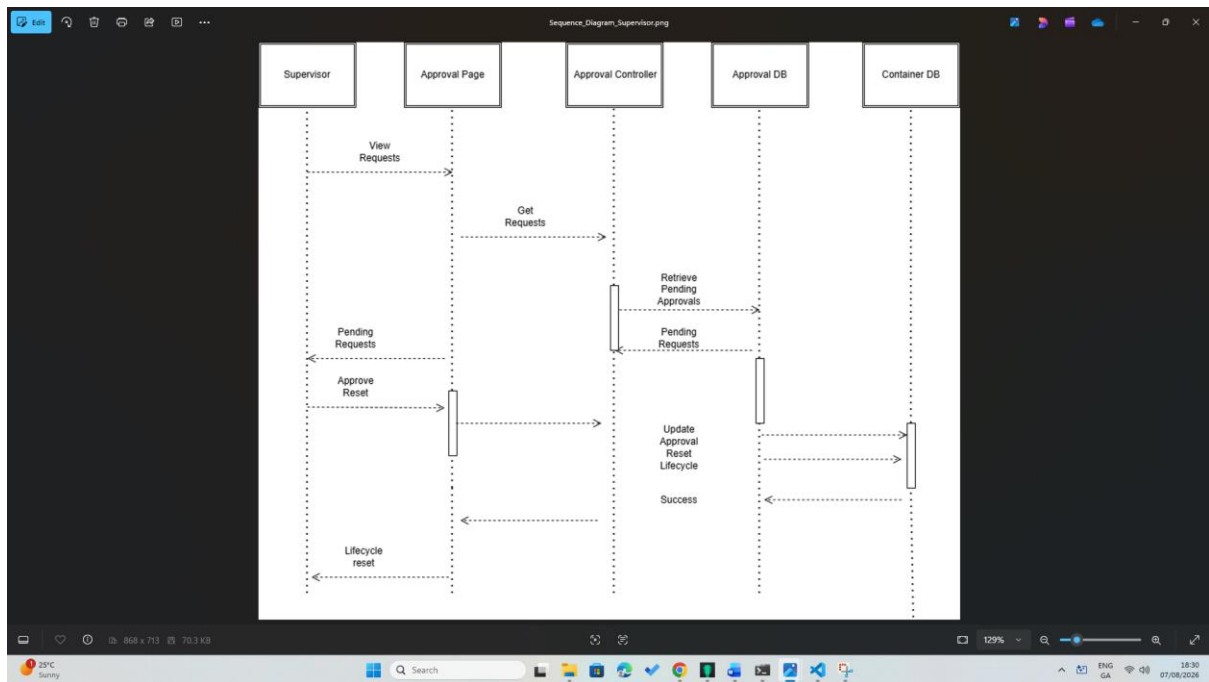
Sequence Diagram of movement



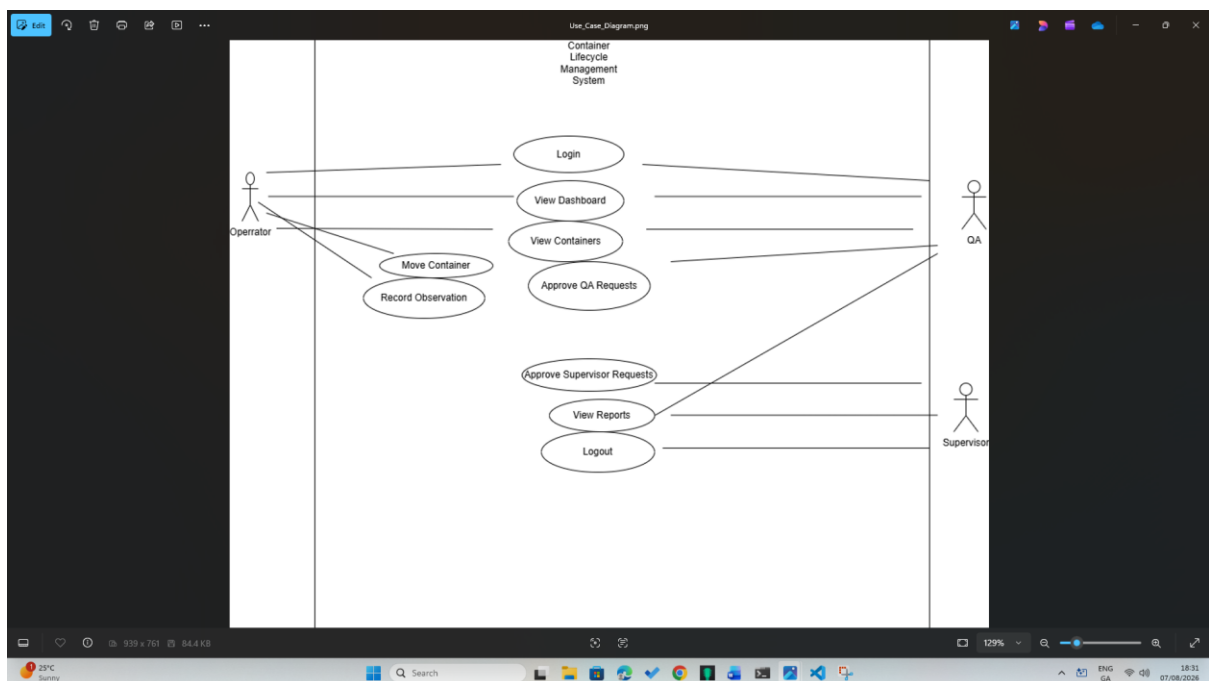
Sequence diagram of Supervisor approval



Sequence Diagram of QA approval



Use case Diagram



The design artefacts presented in this appendix were produced during the analysis and design phases of the project. They provided the foundation for the implementation of the Container Management System by defining the system architecture, database structure, workflows, and user interactions.

Appendix D – Database Design

This appendix provides supporting information relating to the database developed for the Container Management System. The database was implemented using MySQL and was designed to maintain data integrity, support workflow management, and provide complete traceability throughout the container lifecycle.

```

MySQL 8.0 Command Line Client
+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+
| id | int | NO | PRI | NULL | auto_increment |
| container_code | varchar(50) | NO | UNI | NULL | |
| created_at | timestamp | NO | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| current_status | int | NO | MUL | NULL | |
| location_id | int | NO | MUL | NULL | |
| requires_ga_approval | tinyint(1) | YES | | 0 | |
| last_cycle_start_at | datetime | YES | | NULL | |
| initial_ga_approved_at | datetime | YES | | NULL | |
| use_count | int | NO | | 0 | |
| requires_supervisor_reset | tinyint(1) | YES | | 0 | |
+-----+
10 rows in set (0.07 sec)

mysql> describe approval_requests;
+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+
| id | int | NO | PRI | NULL | auto_increment |
| container_id | int | NO | MUL | NULL | |
| from_stage | int | NO | | NULL | |
| to_stage | int | NO | | NULL | |
| requested_action | varchar(100) | NO | | NULL | |
| required_role | varchar(20) | NO | | NULL | |
| comments | text | YES | | NULL | |
| status | enum('PENDING', 'APPROVED', 'REJECTED') | YES | | PENDING | |
| requested_at | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| reviewed_at | timestamp | YES | | NULL | |
| requested_by_user_id | int | NO | MUL | NULL | |
| reviewed_by_user_id | int | YES | MUL | NULL | |
| used_at | datetime | YES | | NULL | |
+-----+
13 rows in set (0.00 sec)

mysql> describe container_movements;
+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+
| id | int | NO | PRI | NULL | auto_increment |
| container_id | int | NO | MUL | NULL | |
| from_stage | int | NO | | NULL | |
| to_stage | int | NO | | NULL | |
| moved_at | datetime | NO | | NULL | |
| moved_by_user_id | int | NO | MUL | NULL | |
| approved_by_user_id | int | YES | MUL | NULL | |
+-----+
7 rows in set (0.00 sec)

mysql>

```

Foreign key relationships enforce referential integrity between the database tables. Each container can have multiple movement records and approval requests, while users may perform many workflow actions. This structure supports complete lifecycle traceability and ensures operational data remains consistent.

Appendix E – API Endpoints

The Container Management System exposes a RESTful API that enables communication between the React frontend and the Node.js backend. The API supports user authentication, container lifecycle management, approval processing, dashboard reporting, analytics, and QR code scanning.

Authentication

Method	Endpoint	Description
POST	/api/auth/login	Authenticate a user and return a JWT token.
GET	/api/auth/me	Retrieve the authenticated user's profile information.

Container management

Method	Endpoint	Description
GET	/api/containers	Retrieve all containers.
GET	/api/containers/:id	Retrieve a specific container.
POST	/api/containers	Register a new container.
PUT	/api/containers/:id/move	Move a container through the workflow.
GET	/api/containers/:id/history	Retrieve the movement history of a container.

Approval management

Method	Endpoint	Description
GET	/api/approvals	Retrieve all pending approval requests.
PUT	/api/approvals/:id	Approve or reject a pending request.

Dashboard and Analytics

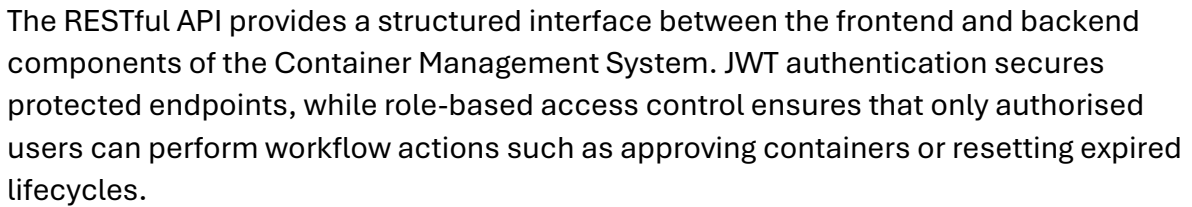
Method	Endpoint	Description
GET	/api/dashboard/summary	Retrieve operational dashboard statistics.
GET	/api/dashboard/activity	Retrieve recent workflow activity.
GET	/api/analytics	Retrieve analytical data for reporting and charts.

Quick scan

Method	Endpoint	Description
GET	/api/containers/scan/:code	Retrieve a container using its QR code or barcode.

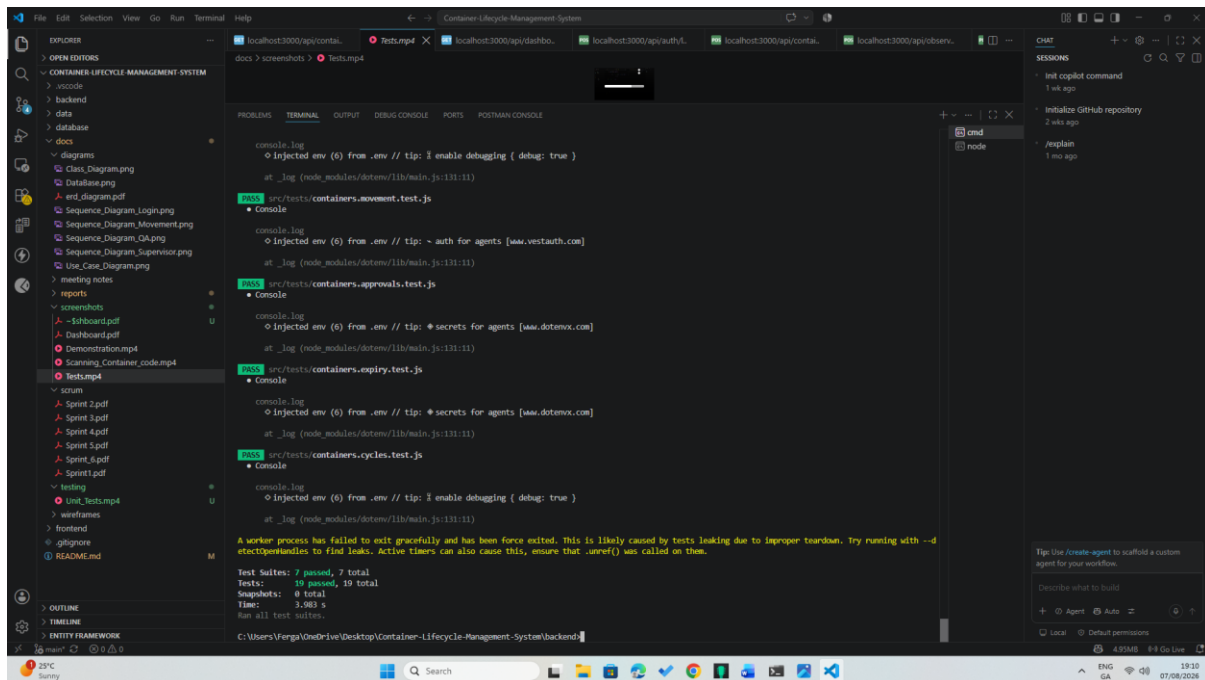
Thunder client operator login





Appendix F – Testing Evidence

The completed Container Management System was tested using a combination of automated and manual testing to verify that the application functioned as expected. Automated testing was performed using Jest, while manual testing was carried out using Thunder Client and the React user interface. Testing focused on validating authentication, workflow management, approval processes, lifecycle rules, and system integration.

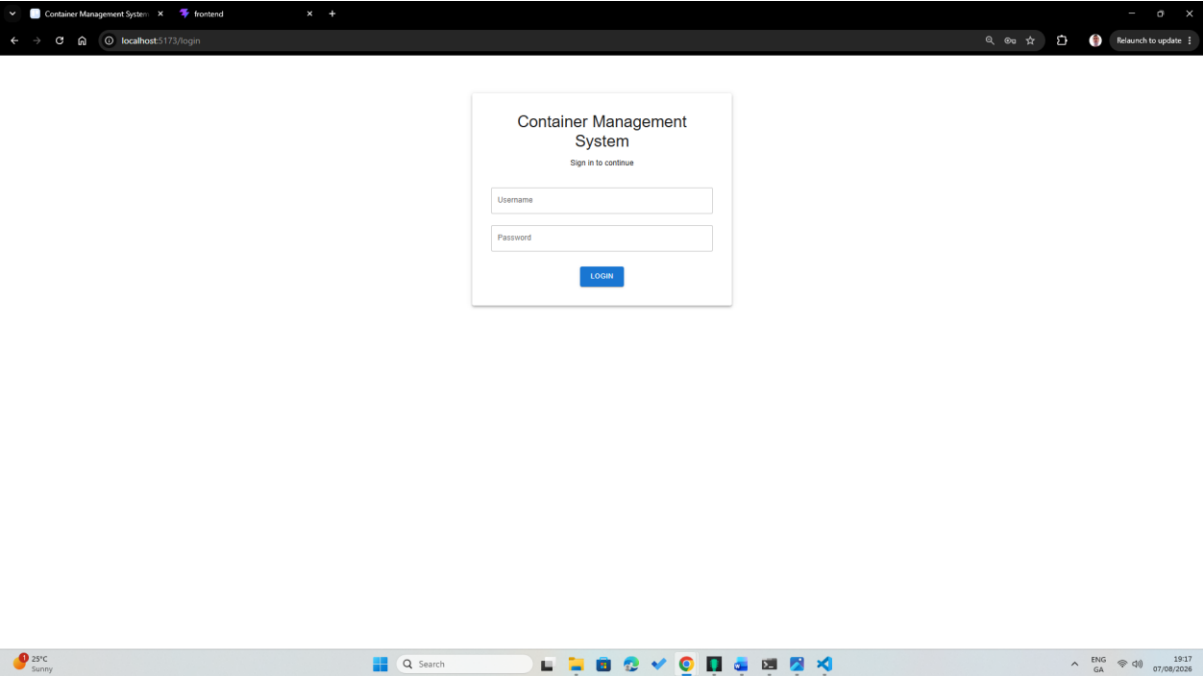


Testing confirmed that the Container Management System successfully implemented the required business rules and functional requirements. Automated testing, manual testing, and end-to-end workflow validation demonstrated that the application consistently enforced lifecycle rules, approval workflows, authentication, and traceability throughout the complete container lifecycle.

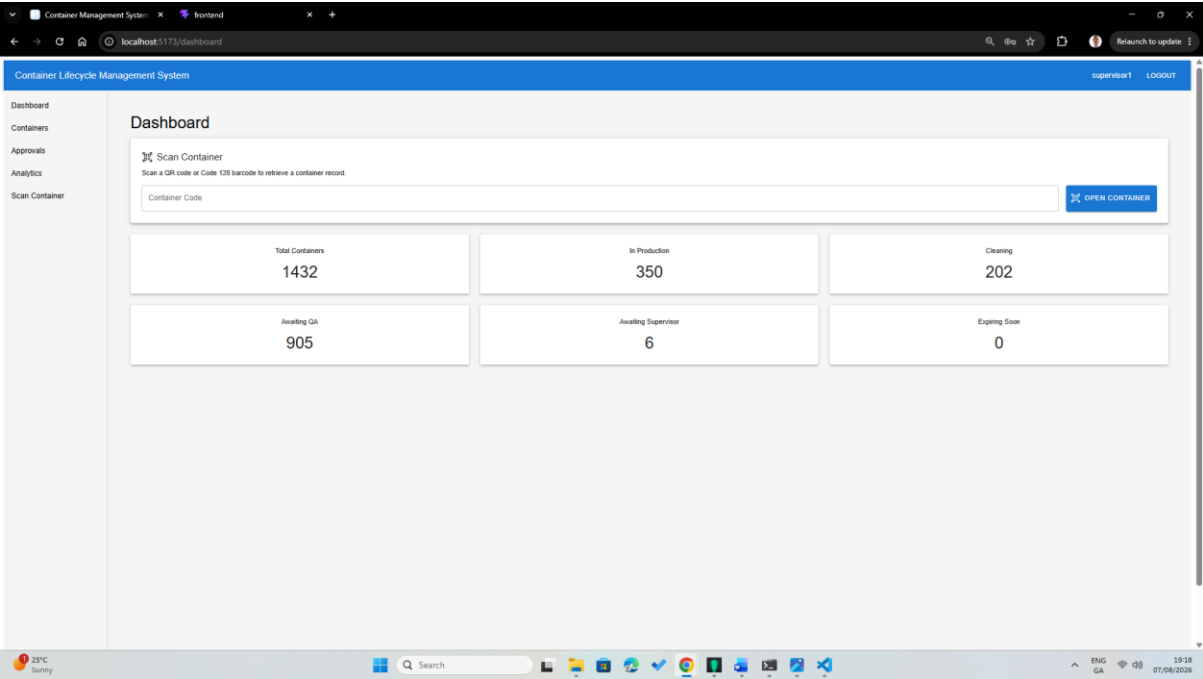
Appendix G – User Interface Screenshots

This appendix presents the main user interfaces developed for the Container Management System. The screenshots demonstrate the completed application and illustrate how users interact with the system to manage reusable production containers throughout their lifecycle.

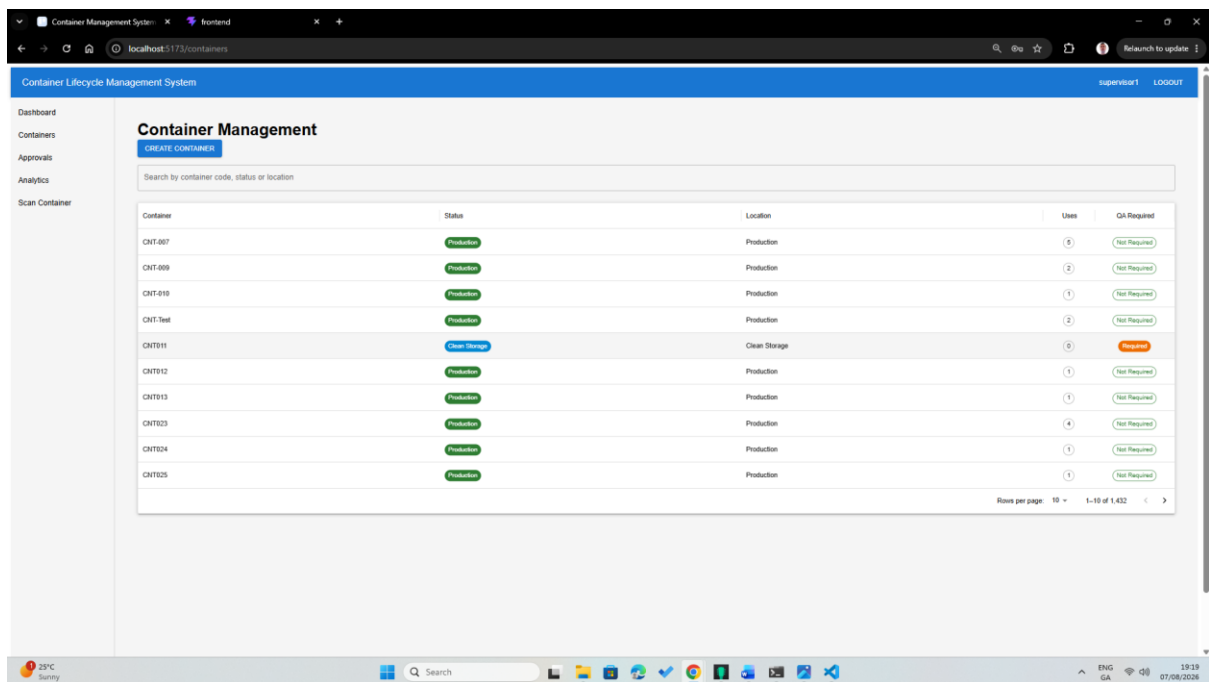
Login



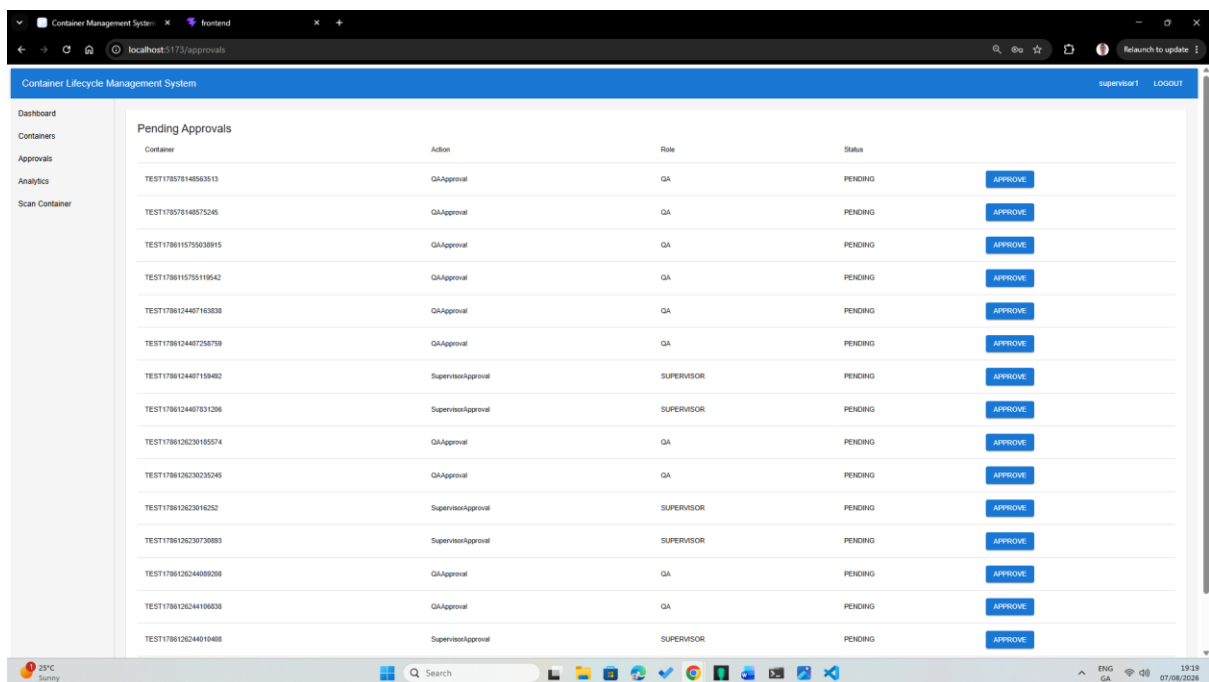
Dashboard



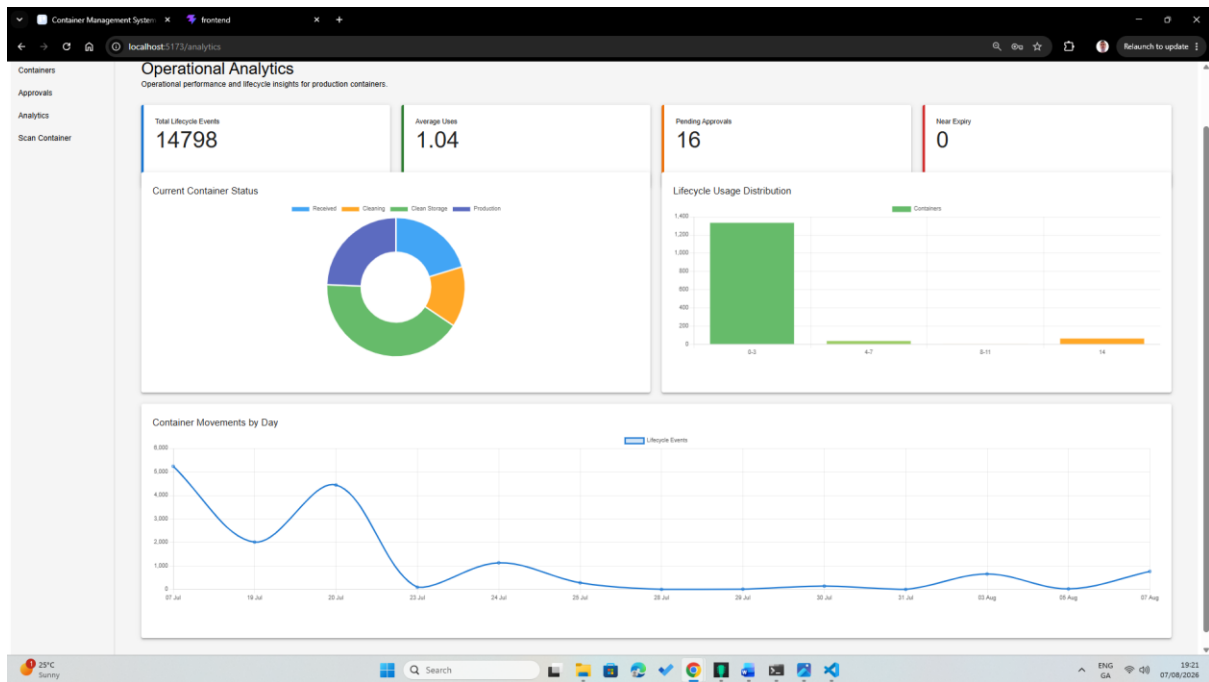
Containers



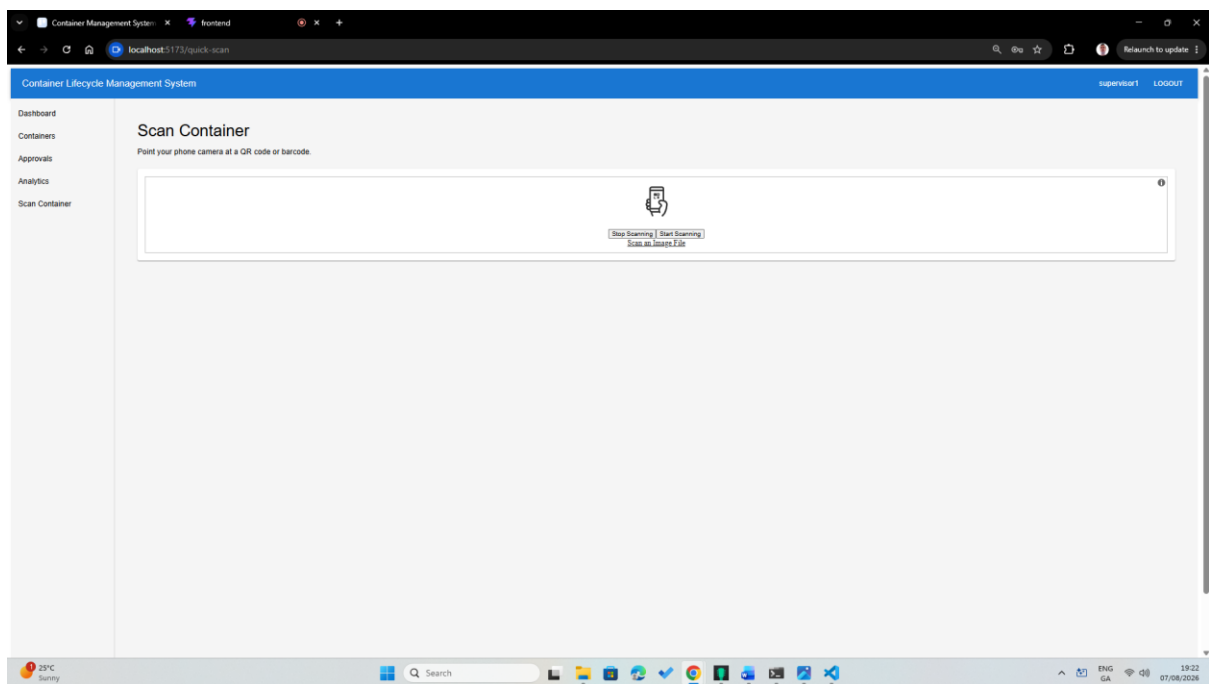
Approvals



Analytics



Quick scan



The completed user interface was developed using React and Material UI, providing a responsive and intuitive environment for managing reusable production containers. The interface supports secure authentication, workflow management, approvals, dashboard reporting, analytics, and QR code scanning, enabling users to efficiently manage the complete container lifecycle.